

Actividad: Colaboración con Git y GitHub

Actividad: Colaboración con Git y GitHub

Paso 1: Prepara el entorno de trabajo del equipo

Paso 2: Crear y subir cambios en la rama personal

Paso 3: Integrar tu rama en main y publicar main remoto

Paso 4: Integración, Colaboración y resolución de conflictos

Push Directo vs. Pull Request

Paso 5: Colaborar con un Pull Request

Descargar

En estas actividades, aprenderéis a colaborar en un mismo proyecto utilizando Git y GitHub. Seguiremos un **flujo de trabajo básico pero fundamental** para cualquier equipo de desarrollo. El objetivo es que os **familiaricéis con los comandos esenciales** y las buenas prácticas para trabajar de forma sincronizada.

Herramientas:

- **Navegador web** (para GitHub)
- **Visual Code** (para los comandos de Git en local)

Paso 1: Prepara el entorno de trabajo del equipo

Organizad el espacio de trabajo del equipo en GitHub como ya hicisteis en la Actividad 2:

- El profesor o vosotros os debéis organizar en grupos de 4 o 5 compañeros.
- Cada grupo deberá decidir quien va a ser el **Owner**.
- Cread una **organización** (por ejemplo, `equipo3_2dam_balmis`), invitad a todos los miembros del grupo y añadid al profesor con rol **Owner**.
- Dentro de esa organización cread un repositorio de trabajo público (por ejemplo, `proyecto-colaborativo`) inicializado con `README.md`, será el repositorio común que usaréis durante toda la actividad.
- Cada miembro del equipo deberá clonar el repositorio remoto en su máquina local y crear su rama personal con su nombre, dejándola asociada al remoto. *Se deja una pista en el siguiente código, pero deberéis de adaptar los comandos.*

```
git clone https://github.com/equipo3_2dam_balmis/proyecto-colaborativo.git
cd proyecto-colaborativo
git switch -c nombre-alumno
git push -u origin nombre-alumno
```

Recuerda

La opción `-u` deja vinculada la rama local con la rama remota para que, a partir de ese momento, podáis usar `git push` y `git pull` sin indicar siempre `origin nombre-alumno`.

Paso 2: Crear y subir cambios en la rama personal

Cada alumno, trabajando en su rama personal, debe crear un archivo con su nombre, añadir contenido, subirlo al remoto y comprobar que su repositorio local queda sincronizado con GitHub.

```
# 1) Asegúrate de estar en tu rama
git switch nombre-alumno

# 2) Crea tu archivo (ejemplo: ana.md) y edítalo
notepad ana.md

# 3) Añade, confirma y sube tus cambios
git add ana.md
git commit -m "Añade archivo personal de Ana"
git push

# 4) descarga información del remoto (sin mezclar) y verifica las diferencias con el arch
git fetch origin
git diff ana origin/ana

# 5) Trae todos los cambios del remoto
git pull
```

Comprobación final: `git status` debe indicar *working tree clean* y no debe haber diferencias pendientes con el remoto de vuestra rama.

Paso 3: Integrar tu rama en main y publicar main remoto

Aunque tenemos actualizada en local y remoto nuestra rama, los cambios no están subidos a la rama main todavía. Por lo que deberemos de realizar los siguientes pasos: primero deberemos integrar en la `main local` los cambios de tu rama personal y después publicarlos en remoto (GitHub).

Warning

Solo puedes hacer `push` si no hay cambios en el remoto que tú no tengas. Si alguien más ha subido algo, primero debes hacer `git pull`.

```
git switch main
git pull origin main
git merge nombre-alumno
git push origin main
git switch nombre-alumno
```

Nota

Este comando coge todos los commits de tu rama personal y los aplica sobre la rama `main` en tu ordenador. Ahora tu `main` local tiene los cambios de tus compañeros (del `pull`) y los tuyos (del `merge`).

Paso 4: Integración, Colaboración y resolución de conflictos

En los anteriores pasos, cada miembro del equipo ha trabajado en un archivo independiente del repositorio, por lo que no ha interferido con los cambios de ningún otro miembro, pero ¿qué pasa si dos personas modifican la misma línea del mismo fichero al mismo tiempo? Git no sabrá qué cambio conservar y se producirá un **conflicto**. Vamos a simularlo.

Preparación:

- **Dos miembros del equipo** (Persona A y Persona B) se ponen de acuerdo.

- Ambos deben tener su rama `main` local actualizada: `git switch main` y luego `git pull origin main`.
- Van a modificar el fichero `README.md`.

Acciones Simultáneas:

1. Persona A:

- Abre `README.md` (`notepad README.md`).
- Modifica la primera línea añadiendo: `Proyecto del Equipo Alfa`.
- Guarda el fichero.
- Ejecuta los siguientes comandos:

```
git add README.md
git commit -m "Modificación de Persona A"
git push origin main
```

- El `push` de la Persona A tendrá éxito. ¡Ha sido más rápida!

2. Persona B:

- Al mismo tiempo, abre `README.md`.
- Modifica la **misma primera línea** añadiendo: `Pruebas de colaboración - Equipo Alfa`.
- Guarda el fichero.
- Ejecuta los siguientes comandos:

```
git add README.md
git commit -m "Modificación de Persona B"
git push origin main
```

El Conflicto:

El `push` de la Persona B **fallará**. Git mostrará un error indicando que el repositorio remoto tiene cambios que no tienes en tu local. Git te sugerirá que hagas un `git pull` primero.

Resolución (Persona B):

1. Intenta sincronizar con `git pull`:

```
git pull origin main
```

2. Git intentará fusionar los cambios remotos con los tuyos, pero fallará y mostrará un mensaje de `MERGE_CONFLICT`. Te dirá en qué fichero está el conflicto (`README.md`).

3. Abre el fichero conflictivo:

```
notepad README.md
```

Verás algo como esto:

```
<<<<<< HEAD
Proyecto del Equipo Alfa
=====
Pruebas de colaboración - Equipo Alfa
>>>>>> [hash-del-commit-remoto]
```

Explicación de los marcadores

- `<<<<<< HEAD` : Indica el inicio de tus cambios locales.
- `=====` : Separa tus cambios de los cambios que vienen del repositorio remoto.
- `>>>>>> ...` : Marca el final de los cambios remotos.

4. **Resuelve el conflicto:** Debes editar el fichero manualmente para decidir cómo debe quedar la versión final. Elimina los marcadores de Git (`<<<` , `===` , `>>>`) y deja el contenido deseado. Por ejemplo, podéis decidir combinar ambas ideas:

```
# Pruebas de colaboración - Proyecto del Equipo Alfa
```

5. **Guarda el fichero** una vez resuelto.
6. **Informa a Git de que el conflicto está resuelto:**

```
git add README.md
```

7. **Completa la fusión con un commit:**

```
git commit
```

Se abrirá un editor de texto con un mensaje de commit predefinido. Simplemente guarda y cierra el editor para confirmar.

8. **Ahora sí, haz el `push` final:**

```
git push origin main
```

¡Éxito! El conflicto ha sido resuelto y los cambios están en GitHub.

Push Directo vs. Pull Request

Aunque en este ejercicio hemos hecho ambas cosas, en entornos profesionales casi siempre se prefiere el Pull Request para la rama principal.

Característica	Push Directo a <code>main</code>	Pull Request (PR)
Velocidad	Ventaja: Muy rápido y directo. Ideal para proyectos personales o correcciones triviales.	Inconveniente: Proceso más lento, requiere más pasos y la intervención de otra persona.
Calidad y Revisión	Inconveniente: No hay revisión de código. Es fácil introducir errores (<code>bugs</code>) en la rama principal.	Ventaja: Permite la revisión por pares . Otros pueden revisar el código, sugerir mejoras y detectar errores antes de que lleguen a <code>main</code> .
Colaboración	Inconveniente: Es un proceso aislado. No fomenta la discusión sobre los cambios.	Ventaja: Es el centro de la colaboración. Permite discutir los cambios, hacer preguntas y mantener un registro de las decisiones tomadas.
Integración Continua	Inconveniente: Puede romper el <code>build</code> o las pruebas automáticas si se sube código defectuoso.	Ventaja: Se pueden ejecutar pruebas automáticas sobre el PR. Si las pruebas fallan, se bloquea la fusión, protegiendo la rama <code>main</code> .
Historial del Proyecto	Ventaja: Genera un historial más lineal y simple.	Inconveniente: Puede generar un historial más "ruidoso" con commits de fusión, aunque existen estrategias para evitarlo (squash, rebase).

Paso 5: Colaborar con un Pull Request

En lugar de subir los cambios directamente a `main`, ahora usaremos un **Pull Request (PR)**. Este es un mecanismo para proponer cambios y pedir que alguien los revise antes de integrarlos.

1. Sube los últimos cambios de tu rama a GitHub:

```
git push origin [tu-nombre]
```

2. Crea el Pull Request en GitHub:

- Ve a la página del repositorio en tu navegador.
- Verás un mensaje amarillo que dice **"Your branch had recent pushes"**. Haz clic en el botón verde **"Compare & pull request"**.
- Se abrirá un formulario. Asegúrate de que la base sea `main` y la rama a comparar sea `[tu-nombre]`.
- Añade un título y una descripción para tu PR.
- En la barra lateral derecha, busca la sección **"Reviewers"** y selecciona a un compañero del equipo para que revise tus cambios.
- Haz clic en **"Create pull request"**.

3. Revisar y Aprobar el Pull Request (acción del revisor):

- El compañero asignado recibirá una notificación.
- Debe ir a la pestaña **"Pull Requests"** del repositorio y abrir tu PR.
- En la pestaña **"Files changed"**, puede ver los cambios que has hecho.
- Si todo está correcto, debe hacer clic en **"Review changes"**, seleccionar **"Approve"** y enviar la revisión.
- Finalmente, el revisor debe hacer clic en el botón verde **"Merge pull request"** y confirmar la fusión. Esto integrará tus cambios en la rama `main` directamente en GitHub.



Regla de oro de equipo:

Protege la rama principal (main o develop): el código nunca entra directamente, siempre a través de una rama de tarea y un Pull Request revisado.

- **Mandamiento 1:** Mantén tus PRs pequeños y enfocados (una tarea = una rama = un PR). Es más fácil de revisar y más rápido de integrar.
- **Mandamiento 2:** Haz fetch y fusiona la rama principal en tu rama de tarea frecuentemente para resolver conflictos localmente antes de abrir el PR.
- **Mandamiento 3:** Respeta la revisión. No fusiones tu propio PR sin la aprobación de un compañero (el "Revisor" del diagrama).